

TagLab Machine Learning using DeepLabV3+

Standard Operating Procedure

SOP by Joel Betteridge

Important note

Within `training.py`, line 129, `dtype` was originally set to `int`. NumPy's plain `int` maps to a 32-bit integer on Windows, whereas on Linux or Mac it's 64-bit, so this line behaves differently depending on the operating system. When training a large dataset, this causes corrupted arithmetic from overflowed, sometimes negative, confusion matrix entries. I have changed this, but when TagLab is next updated, this fix may be reset and need to be reapplied.

Important note 2

Many of these scripts are already in TagLab-main on the super computer, check before re-downloading, you may be able to just paste the prompt in cmd. You can find all the scripts I used within Intern Projects \ Joel , including scripts for pixel-wise statistics and disagreement maps.

Step 1 — Make a project folder and add your existing projects

IMPORTANT — save and copy all files locally (otherwise you must wait hours for OneDrive sync later).

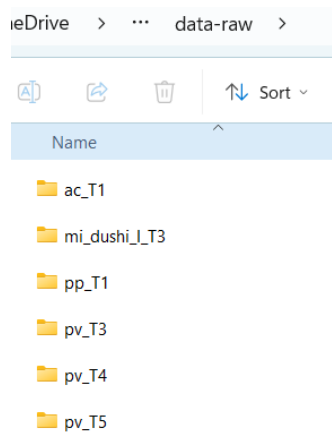
Create a project folder, e.g. `acer_taglab_training`. Within this folder, create two other folders: `data-raw` and `training`. Within `data-raw` you will add your existing projects.

For a previously completed project, find the corresponding `.json` and `.jpg` file and copy them both locally into their site folder within `data-raw`. For example:

`D:/acer_taglab_training/data-raw/ac_T1/COPY_AC_T1.json`

`D:/acer_taglab_training/data-raw/ac_T1/COPY_AC_T1.jpg`

Repeat this for each completed project, like this:



Example: each site's project files copied locally into data-raw

Step 2 — Open one of the projects

Open TagLab and open a project from a site folder within data-raw.

If TagLab can't find the orthomosaic, it will ask you to locate it again, choose the corresponding .jpg file in the site folder.

Step 3 — Look at the Labels panel

On the right-hand side you'll have your classes. For example, yours might look something like:

-  Staghorn
-  Porites
-  Montipora
-  Algae
-  Rubble

We want only 'Staghorn' available.

Click Project → Labels Dictionary Editor

Create a new label, 'Staghorn', with a set RGB (e.g. 255,0,0). **You must use the same RGB code for 'Staghorn' in every project, as TagLab recognizes different colours as different classes when creating a training dataset.**

Highlight all the previously identified Staghorn and apply it all to the new 'Staghorn' label.

Step 4 — Turn OFF unwanted classes

Click the eye icon beside every class except Staghorn. Only Staghorn should be visible when we export the training data.

You want:

-  Staghorn
-  Porites
-  Montipora
-  Algae
-  Rubble

If you accidentally leave Porites visible, for example, TagLab will treat it as another training class, even if only 0% of it is highlighted.

Step 5 — Export a training dataset

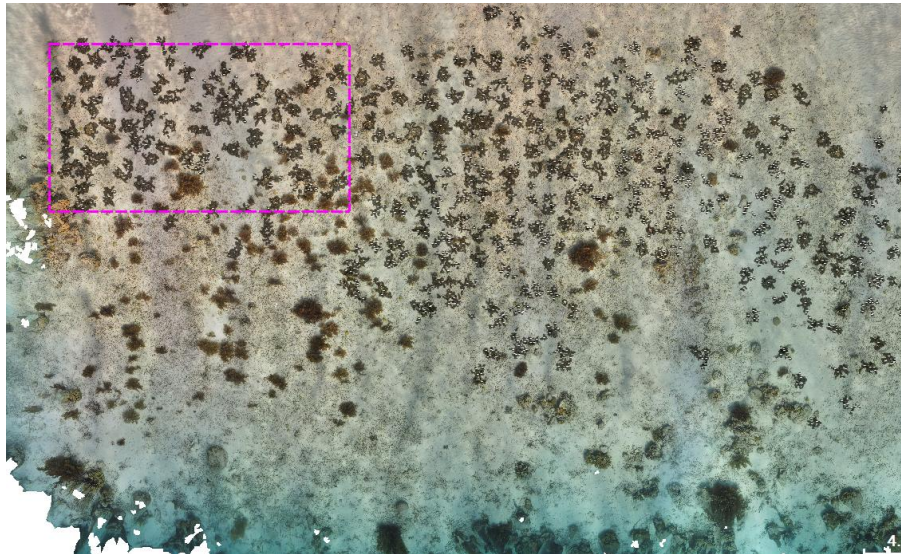
Train → Export New Training Dataset

For dataset folder, enter the 'training' folder you created earlier. Within this folder you need a folder for each site, and within each site, a folder for each working area. For example:

```
D:/acer_taglab_training/training/AC_T1/working_area_1
```

For Working area, select an area where the labels are excellent. Expect to split the entire site into 3–5 smaller working areas. The areas chosen should differ in the type of annotations; the aim is high variability, so the model learns in all environments. Some areas should contain high/low coral density, some with strange angles, some with the target coral close to other species, some with low resolution, etc.

Example working area:



Example working area, selected for variability in annotation conditions

For dataset split: uniform vertical

Target scale is 0.9 or 1 — be consistent each time.

Click export

The working_area_1 folder should now contain three folders and one file: test, training, validation, and target-pixel-size. Do a sanity check by entering the folders and comparing the 'images' and 'labels' folders. Each .jpg in 'images' should have a corresponding .jpg in the 'labels' folder, and the labels version should have the Staghorn highlighted in red (or your RGB of choice).

SAVE THE PROJECT

Repeat for other working areas and sites, including lots of variability. Aim for 20–25 training data sets (including working areas), the more, the better.

Step 6 — Renaming files with PowerShell

Your existing data should currently look like this:

```
training
├── ac_T1
│   └── working_area_1
│       ├── training
│       │   ├── images
│       │   └── labels
│       ├── validation
│       │   ├── images
│       │   └── labels
│       └── test
│           └── images
```

```

├── labels
├── working_area_2
├── ...
├── mi_dushi_I_T3
│   ├── working_area_1
│   ├── working_area_2
│   └── ...
└── other sites...

```

We want one complete folder, containing all sites and working areas, with only 3 folders: 'training', 'validation' and 'test'. However, if we were to copy all images into one folder as-is, file names would overwrite each other.

Create a new folder within 'training' called `Staghorn_v1`.

Inside 'training', shift + right-click the white space and choose 'Open PowerShell window here'.

Paste the merge script in `/scripts/image_merger` into PowerShell and accept any permission prompts.

Step 7 — Check the final report

Check the final report of the script. You want to see something like this:

```

MERGE COMPLETE
Image/label pairs copied: 28743
Missing labels: 0
Duplicate names: 0
Copy errors: 0

```

You can also do a sanity check of the folders and check a few file names to make sure the script worked properly.

Step 8 — Fixing RGBA issues

If you attempted to train the network now, it would crash during 'Computing average', as it would likely hit an image with 4 channels (RGBA) rather than 3 (RGB). This is due to a gap in the working area of the orthomosaic area. This gap can come from the boundaries of the orthomosaic itself, or from empty space within it.

To fix this, we'll convert all RGBA images into RGB images by changing the alpha channel (A) into black, matching the background, and avoiding white, since white is a bleaching indicator.

First, save the script `fix_channel_mismatch.py` in `/scripts`.

Open a command prompt and run:

```
python fix_channel_mismatch.py "D:\Taglab_training\training-data\Staghorn_v1"
```

If the list of images is as expected, re-run with `--fix` added to convert them:

```
python fix_channel_mismatch.py "D:\Taglab_training\training-data\Staghorn_v1" --fix
```

Step 9 — Creating a crash traceback with TagLab

It would be great if training your network worked first time. However, if it crashes while using the standard Start TagLab shortcut, the command window closes before you get a chance to read the error message. From now on, we'll open TagLab manually through cmd, alongside a crash log .txt file.

Open a command prompt and enter:

```
cd C:\TagLab-main
python TagLab.py > crash_log.txt 2>&1
```

This runs TagLab and saves everything it prints into a file called crash_log.txt in the same folder.

Step 10 — Train your network

Open TagLab via Step 9, and open a project with the correct labelling: only Staghorn (255,0,0).

Click Train → Train Your Network

- Network name: Staghorn_v1
- Dataset folder: choose the combined training folder
- Classes to recognize: *
- Training: Preset 1
- Optimizer: Adam
- Number of epochs: **
- Learning rate: 0.00005
- L2 Regularization: 0.0005
- Batch size: ***

** Under 'Classes to recognize', only Staghorn and Background should be present, with a realistic percentage attribution shown — e.g. 95% background, 5% Staghorn.*

*** As a first test run, I recommend around 10 epochs. After a successful run, you can re-run with around 60 epochs.*

**** Batch size is set to 4 by default. You can try this first, but may need to change it later.*

Click Train

Step 11 — Check it reaches 'Training'

It's important not to leave for a dive at this point, crashes will most likely happen during data setup. You won't be able to see the terminal, as output is being sent to crash_log.txt. However, if it crashes, the cmd window will close. If you encounter a crash, open crash_log.txt and troubleshoot appropriately.

Data setup takes around 5 minutes before 'Training — Iteration 1/n' starts. As a rough guide, 100,000 iterations takes around 18 hours.

Step 12 — Sanity check of crash_log.txt

After around 2,000 iterations, open another cmd window and paste this:

```
powershell Get-Content C:\TagLab-main\crash_log.txt -Tail 2000
```

Check for:

- No error messages
- All values are finite and real — you should have no 'nan' values
- Normal noise — iteration values should be bouncing around, not stuck

If things look good, it's time to relax!

Here is the paper on DeepLabV3+, if you're interested in what's happening now: [Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation](#)

Step 13 — Second sanity check of crash_log.txt

The next day, once training is more than halfway through (if it has completed, move on to Step 14), run another sanity check of crash_log.txt (see previous step).

If things are good, keep waiting. If you've run into an error message, troubleshoot accordingly. If you observe a 'nan' cascade, the batch size may be too small.

With batch size 4 and around a 95/5 class imbalance:

- Some batches might be mostly background (nearly zero Staghorn pixels)
- The next batch might be Staghorn-heavy
- This creates wildly inconsistent gradient estimates between batches
- Focal Tversky loss is sensitive to these fluctuations and can explode into nan

You can edit the sanity check command to show the last 10,000 (or more) lines, to find where the 'nan' cascade started.

If the model was still actively learning (batch loss still decreasing substantially), cancel this training run and retry from Step 10 with batch size 8, then 16 if needed. If the 'nan' cascade began after the model had stopped learning (bouncing around a similar batch loss), you can extract the best model and test it.

Note: I created a 98% accurate (Cohen's Kappa) model for staghorn with only 19/40 epochs before nan cascade.

Step 14 — Extraction or completion

If your model needs extracting, proceed with this step. If your model completed, skip to Step 17.

Extraction

Open TagLab-main → models. You should find your model as a .net file (e.g. Stag_v1.net), along with a .txt file of the validation metrics (e.g. Stag_v1-val-metrics.txt). Here is an example of what those metrics look like:

```
CONFUSION MATRIX:
2341318047  32361563
28710916  118386178

NORMALIZED CONFUSION MATRIX:
0.986  0.014
0.195  0.805

ACCURACY      : 0.976
Jaccard Score : 0.000
```

As our training did not reach completion, we must manually compute the 'Average Norm' values (the per-channel mean pixel values from the training images) and add the .net file into the config.json file within TagLab-main.

Step 15 — Computing 'Average Norm'

Open cmd.

Save the script: `compute_dataset_avg.py` within TagLab-main

Run it:

```
cd C:\TagLab-main
python compute_dataset_avg.py
```

Step 16 — Add your .net model into config.json

Open config.json with Notepad++.

Add your model within the file, for example:

```
{
  "Classifier Name": "Staghorn",
  "Weights": "Stag_v1.net",
  "Num. Classes": 2,
  "Classes": {
    "Background": 0,
    "Staghorn": 1
  },
  "Scale": 0.9,
  "Average Norm.": [
    0.445,
    0.4441,
    0.4351
  ]
},
```

Step 17 — Test your first model

Open TagLab along with a new orthomosaic that the model was not trained on. Using a trained ortho will show unrealistically perfect results as it was trained on those (although it does boost your ego).

Click 'Fully automatic semantic segmentation'.

Use the boundary box and preview to explore different areas of the orthomosaic. If you're happy with the model's accuracy, clicking 'Apply' will run the model on the entire orthomosaic. This takes around 40 minutes.

Step 18 — The results and human editing

Observe the results and note areas of weakness in the model. This is the type of content you want to add to the training set to improve its accuracy.

When editing the AI annotations, I recommend using positive/negative clicks segmentation. If the area is large, you should 'cut region' to improve processing efficiency.

Once you are happy, save in-line with the TagLab SOP.

If the model is ready to be retrained (>3 new sites have been completed, follow my other SOP: 'Enhancing the model'.

Stag_v2 is pending ' _'